Oblig 2a – Lineær og logistisk regresjon

Våren 2026

Det er en god idé å lese gjennom hele oppgavesettet før dere setter i gang. Dersom dere har spørsmål så kan dere:

- gå i gruppetime,
- spørre på Discourse
- eller sende epost til in1160-hjelp@ifi.uio.no dersom alternativene over av en eller annen grunn ikke passer for spørsmålet.

Innlevering

Oppgaven leveres innen 11. mars klokken 23.59 i Devilry.

Innleveringen skal bestå av én Jupyter notebook med både kode og tilhørende forklaringer. Før innlevering skal du kjøre gjennom hele notebooken, før du lagrer siste gang. Den bør kjøre uten å feile og vise alt som skal være med.

Vi understreker at innlevering av kode alene ikke er nok for å bestå oppgaven – vi forventer at notebooken også skal inneholde kommentarer (på norsk eller engelsk) på hva dere har gjort og begrunnelser for valgene dere har tatt underveis. La enhver oblig bli en trening i å formidle forskning. Bruk helst hele setninger, og matematiske formler om nødvendig. Resultater skal presenteres i tabeller på en oversiktlig måte. Det å forklare med egne ord, bruke begreper vi har gått gjennom på forelesningene og å forklare og reflektere over løsningene deres er en viktig del av læringsprosessen – ta det på alvor!

Når det gjelder bruk av generative prateroboter (ChatGPT og lignende): Dere kan bruke dem som en "sparringspartner", for eksempel for å forklare noe dere ikke helt har forstått. Dere har imidlertid ikke lov til å bruke dem til å generere løsninger (enten delvis eller fullstendig) til noen av oppgavene. Funksjoner for automatisk skriving av kode, som Copilot i VS Code, må derfor også være deaktivert mens dere jobber på obligen.

Bruker dere KI-verktøy vil vi også at dere kort beskriver hvordan dere har brukt dem under arbeidet med oppgaven.

Det er ikke mulighet for omlevering av obliger som ikke bestås.

Poeng: Obligen gir maksimalt 25 poeng og 6 bonuspoeng. For å bestå kreves det totalt 30 av 50 poeng fra oblig 2a og 2b tilsammen. Bonuspoengene telles ikke med i denne beregningen. Poengfordelingen er markert i overskriften til oppgaven, der `p` indikerer vanlig poeng og `b` indikerer bonuspoeng.

Bakgrunn

I denne obligen skal vi jobbe med lineær og logistisk regresjon. Disse modellene er noen av de nyttigste maskinlæringsmodellene som finnes, der lineær regresjon blir brukt for *regresjon* og

logistisk regresjon blir brukt til *klassifikasjon*. Oppgavene går ut på å først bli kjent med datasetene som blir brukt, og deretter implemetere litt funksjonalitet som tapsfunksjoner og prediksjon med modellene. For å slippe å implementere treningen av modellene skal vi bl.a. bruke modeller fra biblioteket `sklearn` til å trene på datasetene. Til slutt skal resultatet tolkes og det gjøres også litt trekk-seleksjon (feature selection).

Obligen er delt opp i mange mindre deloppgaver og det følger med tester for alle implementasjonsoppgavene. Testene printer ut feilmeldinger dersom testen feiler, og printer ikke ut noe om testen bestås (med mindre dere sender med argumentet `message_on_pass=True`). Feilmeldingene sjekker for flere typer feil, som feil returverdi, feil utregning, error under kjøring og liknende, så det kan være nyttig å lese feilmeldingen for å hjelpe til med implementeringen av oppgavene. Testene må bestås for at oppgavene skal bli godkjent, men beståtte tester garanterer ikke at implementasjonen er korrekt.

Oppgavene skal implementeres med NumPy og klassene `LinearRegression` og `LogisticRegression` fra `sklearn`. Det er ikke tillatt å importere funksjoner som gjør det oppgaven ber dere om å implementere, for eksempel å bruke `accuracy_score` fra `sklearn` for å implementere `calculate_accuracy`.

Vi starter med å importere de nødvendige bibliotekene:

```
import numpy as np
from sklearn.linear_model import LinearRegression, LogisticRegression

from tests2a import (
    test_calculate_accuracy,
    test_calculate_bce,
    test_calculate_mse,
    test_predict_linear_regression,
    test_predict_logistic_regression,
    test_sigmoid,
)
from utils2a import get_auto_mpg_data, get_spambase_data
```

1 - Lineær regresjon [Totalt 12p og 3b]

1.0 - Bakgrunn og dataset [2p, 1b]

Lineær regresjon er en enkel, men effektiv, regresjonmodell. Modellen predikerer et reelt tall (fra minus uendelig til pluss uendelig) gitt en liste med inputtrekk (features). Ved å bruke et datasett som inkluderer både input og tilhørende fasitverdier kan vi finne koeffisientene som minimerer forskjellen mellom prediksjonene og fasitverdiene.

Vi starter med å se på datasettet vårt. Vi skal bruke `Auto MPG` datasettet, som er et mye brukt og konseptuelt relativt enkelt datasett for regresjon. Det inneholder informasjon om biler fra 70- og 80-tallet, som blir brukt til å predikere hvor bensineffektive bilene er, målt i *miles per gallon* (mpg). Vi skal først jobbe med to trekk: `weight`, som er vekten til bilene, og `acceleration`, som er hvor rask bilene kan akselerere. Litt senere i obligen vil vi inkludere flere trekk fra datasettet.

La oss først avklare notasjonen litt: Vi bruker n til å notere antall datapunkter i datasettet, som er 398 for datasettet vårt, eller 392 etter at vi har fjernet datapunkter med manglende verdier. Hvert av datapunktene har input x og sann verdi (target) y. Dette vil si vi har to lister av lengde n, input $\mathrm{x}_1, \mathrm{x}_2, \ldots, \mathrm{x}_n$ og de sanne verdiene $y_1, y_2, \ldots, y_n$. Merk at input x er notert med **fet skrift**, som representerer en vektor med flere tall. Vi starter med $p=2$ antall trekk, som for bil nummer i notert ved $\mathrm{x}_i = (x_{i,1}, x_{i,2})$, der $x_{i,1}$ representerer vekten til bil nummer i og $x_{i,2}$ representerer bil nummer i sin akselerasjon. Vi bruker ofte indeksen i til å representere et *vilkårlig* tall, som her kan betegne et tall mellom 1 og n.

Vi starter med å laste inn datasettet. Datasettet blir lest med biblioteket `seaborn`. Dere skal ikke endre på noen av argumentene i funksjonen under.

```
mpg_data = get_auto_mpg_data(
    columns_to_include=["weight", "acceleration"],
    val_ratio=0.2,
    test_ratio=0.2,
    perform_scaling=False,
)
x_train = mpg_data["x_train"]
y_train = mpg_data["y_train"]
```

Merk: Dataen over er ikke skalert (fra keyword argumentetet `perform_scaling=False`). Dette er for å bli bedre kjent med dataen i menneskelige lesbare måleenheter. Når vi bruker dataen til å trene og predikere senere er det viktig at dataen blir skalert, ved å gi inn argumentet `perform_scaling=True`.

Objektet som blir returnert, `mpg_data`, er en Python ordbok (dictionary) med trenings-, validerings- og testdatasplitter, samt navn på trekkene. Som vanlig kommer vi til å trene modellen med treningsdataene, bruke evalueringsdataene til å finne ut hvilke treningskonfigurasjoner som fungerer best, og spare testdataene helt til slutt for å gjøre et estimat på hvor bra modellen vår vil fungere. Til å begynne med skal vi kun se på treningsdataene.

Input-dataene, gitt ved `mpg_data["x_train"]`, er et to-dimensjonalt NumPy array. Radene representerer ulike datapunkter og kolonnene forskjellige trekk. Dere kan indeksere rad i og kolonne j med `x_train[i, j]`. NumPy-indeksering har mange nyttige funksjoner. I tillegg til å indeksere en enkel verdi, kan man *slice* en delmengde av arrayen. For å få kolonne nummer 1 for rad 3 til rad 8 kan man for eksempel bruke `x_train[3:9, 1]`, og for å få kolonne 0 for alle radene kan man bruke `x_train[:, 0]`.

La oss gjøre noen oppgaver for å bli kjent med datasettet og NumPy-indeksering. Alle oppgavene forholder seg til treningsdatasplitten for Auto MPG og indeksering starter på 0. Alle svarene kan finnes ved NumPy-indeksering og funksjoner uten looping. Print ut svarene på et lesbart og pent format. Verdien til `mpg_data["features_names"]` har navn tilsvarende kolonnene i x-dataene (her er `weight` den nulte kolonnen og `acceleration` den første). Måleenhenten til vekt er "pounds" og måleenheten til akselerasjonen er antall sekunder bilen bruker på 0 til 60 "miles per hour".

Oppgave 1.0:

- a) Hvor mye veier bil nummer 3?
- b) Hva er akselerasjonen til bil nummer 100?
- c) Hva er gjennomsnittlig vekt for bilene og gjennomsnittlig akselerasjon?
- d) Hva er det høyeste og laveste bensinforbruket (oppgitt i "miles per gallon")?
- e) (*Bonus*) Hva er gjennomsnittlig bensinforbruk for bilene som veier mer enn 3000 pounds?

(Skriv svarene dine her)

```
# x_train, y_train er allerede lastet inn fra mpg_data
# x_train = mpg_data["x_train"]
# y_train = mpg_data["y_train"]

# Oppgave 1.0 a) Hvor mye veier bil nummer 3?
# rad 3 sin kolonne plass 0 representerer bil 3 sin vekt
# x_train er en 2D-array med shape(N, 2): kolonne 0=weight(pounds) og
kolonne 1=acceleration (0-60mph)
car_3_weight = x_train[3, 0]

# Oppgave b) Hva er akselerasjonen til bil nummer 100?
# rad 100 sin kolonne plass 1 representerer bil 100 sin akselerasjon
car_100_accel = x_train[100, 1]

# Oppgave c) Hva er gjennomsnittlig vekt for bilene og gjennomsnittlig
akselerasjon?
# Gjennomsnittlig vekt er: alle radene (alle bilene) sine første
kolonne (vekter); .mean()
mean_weight = x_train[:, 0].mean()
# Gjennomsnittlig akselerasjon for alle biler: alle radene og dem
andre kolonnen (akselerasjonsverider); .mean()
mean_accel = x_train[:, 1].mean()

# Oppgave d) Hva er det høyeste og laveste bensinforbruket (oppgitt i
"miles per gallon")?
# y_train er en 1D-array med mpg (miles per gallon)
max_mpg = y_train.max()
min_mpg = y_train.min()

# Oppgave e) Hva er gjennomsnittlig bensinforbruk for bilene som veier
mer enn 3000 pounds?
# x_train[:, 0] gir alle vektene (kolonne 0 = weight)
# Så sammenligner vi hver vekt med 3000 -> gir en boolsk array med
True/False, dette kalles en boolsk maske.
heavy_3000 = x_train[:, 0] > 3000
# y_train inneholder mpg-verdiene, og rad med indeks "i" i x_train
matcher samme rad i y_train. Når vi så bruker den boolske masken fra
x_train på y_train: y_train[heavy_3000], så får vi kun mpg-verdiene
for biler som veier mer enn 3000 pounds.
# Til slutt tar vi gjennomsnittet (mean) av disse mpg-verdiene
mean_3000_mpg = y_train[heavy_3000].mean()
```

```python
# Her er fine prints:
print("Auto MPG - Train split:")
print("-" * 50)
print(f"a) Car 3 weight:            {car_3_weight:.1f} pounds")
print(f"b) Car 100 acceleration:    {car_100_accel:.2f} seconds")
print()
print(f"c) Average (mean) weight:   {mean_weight:.1f} pounds")
print(f"   Average (mean) accel:    {mean_accel:.2f} seconds")
print()
print(f"d) Max miles per gallon:    {max_mpg:.2f} miles/gallon")
print(f"   Min miles per gallon:    {min_mpg:.2f} miles/gallon")
print()
print(f"e) Mean mpg (weight>3000):  {mean_3000_mpg:.2f} miles/gallon")
print(f"   Count (weight>3000): {heavy_3000.sum()} cars")
```

```
Auto MPG - Train split:
--------------------------------------------------
a) Car 3 weight:            4380.0 pounds
b) Car 100 acceleration:    15.40 seconds

c) Average (mean) weight:   2965.1 pounds
   Average (mean) accel:    15.61 seconds

d) Max miles per gallon:    46.60 miles/gallon
   Min miles per gallon:    9.00 miles/gallon

e) Mean mpg (weight>3000):  17.11 miles/gallon
   Count (weight>3000): 98 cars
```

1.1 - Gjennomsnittlig kvadratfeil (mean squared error) [2p]

Modellen finner verdien på koeffisienten ved å minimere feilen på treningsdataene. Feilen måles mellom prediksjonene $\hat{\mathrm{y}}$ (uttalt *y-hatt*) og fasitverdiene y.

Det er generelt mange måter å måle feil og forskjeller på, men for lineær regresjon bruker man som regel *gjennomsnittlig kvadratfeil*, som regel kalt *mean squared error* (MSE). Dette måler altså den gjennomsnittlige kvadratiske forskjellen mellom prediksjonene og fasitverdiene, altså:

$$\mathrm{MSE}(\mathrm{y}, \hat{\mathrm{y}}) = \frac{1}{n} \sum_{i=0}^{n} (y_i - \hat{y}_i)^2$$

Notasjon: Symbolet Σ er den store greske bokstaven *sigma*, som representerer en sum. Her blir $\sum_{i=0}^{n} (y_i - \hat{y}_i)^2 = (y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + \ldots + (y_n - \hat{y}_n)^2$.

Vektorisering: Hele obligen kan løses enten med eller uten vektorisering. Vektorisering handler om å la lavnivå-biblioteker som NumPy håndtere iterering (løkker), istedenfor å loope eksplisitt i Python. Vektorisering gir mye raskere kode, siden itereringen skjer i C istedenfor Python, i tillegg til at bibliotekene kan håndtere passende datastrukturer, minneallokering og parallelisering.

Koden blir også mer kortfattet, siden man kan erstatte løkker med funksjonskall. Dette kan både gjøre koden mer lesbar, men også føre til at det er vanskeligere å forstå hva koden egentlig gjør. Dere kan selv velge om dere vil bruke NumPy funksjoner (som `np.square()`, `np.mean()`, vektorprodukt og matrisemultiplikasjon) eller om dere vil iterere eksplisitt. Det er også mulig å vektorisere deler av koden, eller først skrive koden med løkker og deretter erstatte den med vektorisert kode. Dette er en god måte å lære å skrive effektiv kode og bedre forstå hva de vektoriserte funksjonene gjør.

Notater

Hva betyr MSE og hva står de forskjellige delen av formeln av:

- y_i står for den ekte verdien.
- y-hatt_i står for den predikerte verdien.
- (y_i - y-hatt_i) er feil-verdien.
- Vi kvadrerer feilen, som gjør den positiv og også straffer større feil mer enn små feil.
- Summerer alle feilene.
- Deler på antall datapunkter: n. Altså:
- MSE = Gjennomsnittet av de kvadrerte feilene som oppstår.

MSE må:

- Returnere en float.
- Fungere for NumPy-arrays
- Fungere uten loops (i oblig tilfelelt her hvertfall).
- Ikke bruke sklearn-funksjoner (--||--).

Oppgave 1.1: Implementer funksjonen `calculate_mse()`, som tar to arrayer av samme lengde og regner ut og returnerer MSE-en.

```
def calculate_mse(y_data, predictions):
    """
    Calculates and returns the mean squared error (MSE).
    Except two arrays of the same length as input.

    Args:
        y_data (np.array): The true values as an array of numbers.
        predictions (np.array): The predictions as an array (of same
length as `y_data`) of numbers.

    Returns:
        float: Value corresponding to the MSE.
    """
    # God praksis å sjekke om input argumentene har lik lengde
    if len(y_data) != len(predictions):
        raise ValueError("y_data and predictions must have the same
lenght!")

    # Vi kalkulerer residualene (feil-verdiene).
```

```
    # residuals er forskjellen mellom de sanne og predikerte verdiene
    residuals = y_data - predictions

    # Vi kvadrerer residual-verdiene, slik at alle verdiene er
positive og store feil straffes hardere. Hvis vi ikke kvadrer dem så
vil positive og negative verdier kansellere hverandre ut.
    squared_residuals = residuals**2

    # Finner mean squared verdier for errors, som da er: MSE
    mse = np.mean(squared_residuals)

    return mse

y_true = np.array([2, 4, 6])
y_predicated = np.array([1, 5, 7])

# Expected:
# (2-1)^2 = 1
# (4-5)^2 = 1
# (6-7)^2 = 1
# MSE = (1 + 1 +1)/3 = 1

print(calculate_mse(y_true, y_predicated))

test_calculate_mse(input_function=calculate_mse)

1.0
```

Tester: Obligen inneholder testfunksjoner lagret i `test2a.py` som dere kan bruke til å teste implementasjonene deres. Beståtte tester vil ikke garantere at det ikke er noen som er feil, men dersom noen av testene feiler er det sikkert at noe ikke er riktig. Testene er importert og kalt i prekoden. De tar inn funksjonen de skal teste som argument og itererer over flere test-tilfeller. Dersom testene bestås vil de av konvensjon ikke gi noen output, men dersom dere har lyst på en bekreftelse at testene bestod kan dere sende med argumentet `message_on_pass=True`. Dere står også fritt til å prøve funksjonene selv og lage egne tester, men pass på at den endelige besvarelsen ikke inneholder for mye kode som ikke er direkte svar på oppgaven.

1.2 - Prediksjon [2p]

Vi kan nå implementere prediksjon for lineær regresjon. La oss starte med å visualisere lineær regresjon med figuren under.

Modellen får x som input og gir en prediksjon av den sanne verdien som output, notert ved $\hat{y}$. Med datasettet vårt får den altså bilenes vekt og akselerasjon som input og bruker dette til å predikere hvor mye bensin den bruker. Dette gjør den ved å gange inputet med *vekter* og plusse på et *konstantledd* (også kalt bias). For lineære modeller er vektene og konstantleddet ofte kalt *koeffisienter*, og vi bruker den grenske bokstaven β (uttalt *beta*) til å representere dem. For et datapunkt i, vil modellen vår altså regne ut prediksjonen $\hat{y}_i = \beta_1 x_{i,1} + \beta_2 x_{i,2} + \beta_0$. Her er altså β_1 og

β_2 vektene (eller koeffisientene) til henholdsvis bilenes vekt og akselerasjon, mens β_0 er konstantleddet (biaset). Koeffisientene er altså tall som blir ganget med inputet, som også er tall, og plusset sammen.

La oss implementere prediksjonen for lineær regresjon i en funksjon.

Oppgave 1.2: Implementer `linear_regression_predict()`. Funksjonen tar to argumenter, inputdataene og koeffisientene, og returnerer prediksjonene. Inputdataene er her en to-dimensjonal array av størrelse $[n, p]$, altså n rader og p kolonner, der hver rad tilsvarer et datapunkt og hver kolonne tilsvarer et trekk. Koeffisientene blir representert av en array med $p+1$ verdier, der den nulte verdien er konstantleddet, mens elementet på indeks 1 er vekten for det første trekket, elementet på indeks 2 er vekten til det andre trekket, og så videre. Koeffisientene blir brukt på alle n datapunktene og returnerer en array av lengde n med tilsvarende prediksjoner.

Her er det forskjellige muligheter med hensyn på vektorisering. Vi må lage to (implisitte eller eksplisitte) løkker: en over datapunktene og en over trekkene. Det er mulig å vektorisere begge disse løkkene, man dere kan også iterere eksplisitt over begge, eller gjøre en kombinasjon. For best læringsutbytte kan det være lurt å første iterere eksplisitt og så prøve å erstatte det med vektorisert kode. Operatoren `@` kan brukes som matrisemultiplikasjon med NumPy-arrayer og `np.dot()` kan brukes for å få prikkproduktet, der `np.dot(a, b)` tilsvarer $\sum_{i=1}^{n} a_1 \cdot b_1$.

Notater

Lineær regresjon delt opp:

Beta = B

- B_0 = bias (konstantledd)
- B_1 ... B_p = vektene.
- x_ij er feature j for datapunkt i.
- y-hatt_i er prediksjon for datapunkt i.

Dette skal implementeres for alle datapunktene samtidig.

x_data @ weights:

Hvis vi har:

- x_data = [n, p]
- weights = [p] så:
- x_data @ weights gi følgende:
- For hver rad i x_data så multipliserer vi hver feature med tilhørende vekt og summerer til slutt alt sammen, NumPy gjør dette for oss for alle radene samtidig.

```
def linear_regression_predict(x_data, coefficients):
    """
    Given input data `x_data` of shape [n, p] (`n` datapoints and `p`
features)
```

```
    and `coefficients` of shape [p + 1], returns the predictions the
model
    gives as an [n]-length array.

    Args:
        x_data (np.array): [n, p]-shaped array of input data.
        coefficient (np.array): [p + 1]-shaped array of coefficient.
Element number
            zero corresponds to the bias while element one to p
corresponds to
            the weight for feature 1 to p.

    Returns:
        np.array: [n]-shaped data of corresponding predictions.
    """

    # Sjekker at x_data er 2-dimensjonal med n datapunkter og p
features. ndim = Number of Dimensions
    if x_data.ndim != 2:
        raise ValueError("x_data must be a 2D array with shape [n,
p]!")

    # Henter ut antall datapunkter n og antall features p
    n, p = x_data.shape

    # Vi må ha p+1 koeffisienter, der det er 1 bias + 1 vekt per
feature
    if len(coefficients) != p + 1:
        raise ValueError(f"coefficients must have length p + 1 =
{p+1}, but got {len(coefficients)}!")

    # Første element i koeffisientene er bias, altså konstantleddet
    bias = coefficients[0]

    # Resten av elementene i koeffisientene er vektene til feature
1...p
    weights = coefficients[1:]

    # Vi behandler det matematisk slik:
    # predictions = bias + X * weights
    # x_data @ weights betyr: for hver rad i x_data:
        # Regn ut prikkproduktet (dot) mellom raden og weights
    # Resultatet blir da et array med én verdi per datapunkt med shape
[n].
    predictions = bias + (x_data @ weights)

    return predictions

# eget mini eksempel
x = np.array([
```

```
    [10, 2],
    [3, 7]
])

coefficients_2 = np.array([1, 0.5, -2])
# bias = 1
# weight1 = 0.5
# weight2 = -2

# Første rad:
# y = 1 + 0.5 * 10 + (-2)*2 = 2
# Andre rad:
# y = 1 + 0.5 * 3 + (-2)*7 = -11.5

print(linear_regression_predict(x, coefficients_2))

test_predict_linear_regression(input_function=linear_regression_predic
t)
```

```
[  2.  -11.5]
```

1.3 - Kjøring og evaluering av lineær regresjon [2p]

Vi skal nå bruke maskinlæringsbiblioteket `sklearn`, og mer spesifikt klassen `LinearRegression`, til å trene en regresjonsmodell på datasettet vårt. Modellen kan initialiseres med `model = LinearRegression()`. Deretter kan den trenes ved å kalle `model.fit(X=x_train, y=y_train)`, som vil finne koeffisientene som gjør prediksjonene for treningsdataen `x_train` så lik de sanne verdiene `y_train` som mulig. Deretter kan man bruke modellen til å predikere med `model.predict(X=x_val)`.

Oppgave 1.3: Tren modellen på treningsdataene og prediker $\hat{y}$-verdier for valideringsdataene. Bruk så `calculate_mse` til å regne ut gjennomsnittlig kvadratfeil mellom disse prediksjonene og de sanne verdiene i `y_val`. Rapporter resultatet på et leselig format.

```
# Laster inn Auto MPG-data med scaling aktivert. Siden perform-
scaling=True, så betyr det at features blir skalert.
# Grunnen til at dette er hensiktsmessig akkurat nå er at features som
har store størrelser (f.eks: weight mellom 1000-5000 og acceleration
mellom 8-25) ikke får for stor eller liten påvirkning i modellen som
lages.
mpg_data = get_auto_mpg_data(
    columns_to_include=["weight", "acceleration"],
    val_ratio=0.2,
    test_ratio=0.2,
    perform_scaling=True,
)

# Dette er altså treningsdataen som brukes for å finne koeffisientene
x_train = mpg_data["x_train"]   # Shape blir her: [n, p]
```

```python
y_train = mpg_data["y_train"]  # Shape blir her: [n]

# Dette er valideringsdataen som brukes for å evaluere hvor bra
modellen generaliserer
x_val = mpg_data["x_val"]  # Shape blir her: [n, p]
y_val = mpg_data["y_val"]  # Shape blir her: [n]

# Her lager vi den lineære regresjonsmodellen fra sklearn.
# LinearRegression() funksjonen i sklearn lærer en modeller av typen
y-hatt, altså med bias + en lineær kombinasjon av feature.
model = LinearRegression()

# Nå trener vi modellen med fit().
# model.fit(X, y) finner koeffisientene (beta-verdiene) som minimerer
mse på treningssettet.
# Notat: Dette kan løses med lineær algebra, men sklearn gjør det for
oss nå.
model.fit(x_train, y_train)

# Nå som fit() er brukt, så har vi koeffisientene vi trenger:
# - model.intercept = B_0 (bias)
# - model.coef = [B_1, B_2, ..., B_p] (vektene)
intercept = model.intercept_
coefficients = model.coef_

# Nå predikerer vi på valideringsdataene.
# Bruker modellen model til å lage prediksjoner y_hatt på x_value.
Dermed blir output en 1D-array med samme lengde som y_value
y_val_predicated = model.predict(x_val)

# Nå evaluerer vi MSE på valideringssettet.
# MSE måler jo gjennomsnittet kvadrert feil mellom fasiten som er
y_val og de predikerte verdiene som er y_val_predicated
mse_val = calculate_mse(y_val, y_val_predicated)

# Printer svarene på en fin måte:
print("Lineaer Regression (sklearn) - Evaluation")
print("-"*50)

# Antall datapunkter og features vi har, der p = antall kolonner i x
print(f"Training set: n={x_train.shape[0]}, p={x_train.shape[1]}")
print(f"Value set: n={x_val.shape[0]}, p={x_val.shape[1]}")
print()

# Printer hvilke koeffisienter modellen finner, der Intercept = bias,
Coef = vektene til de skalerte feature-ene
print(f"Intercept (B_0): {intercept:.4f}")
print(f"Coefficients (B_1 ... B_p): {coefficients}")
print()
```

```
# Endelige Resultatet
print(f"Validation MSE value: {mse_val:.4f}")
```

```
Lineaer Regression (sklearn) - Evaluation
--------------------------------------------------
Training set: n=235, p=2
Value set: n=78, p=2

Intercept (B_0): 23.7702
Coefficients (B_1 ... B_p): [-6.24212917  0.99188927]

Validation MSE value: 18.2651
```

Tolkningsnotat av validerings resultatet for lineær regresjon med sklearn

Modellen over er trent med LinearRegression() funksjonen på treningsdataene og har lært med dette:

- y_hat = B_0 + B_1 * weight + B_2 * acceleration

Siden perform_scaling=True, så er både weight og acceleration standardisert (med et typisk mean på 0 og standard deviation på 1). Dette betyr altså at koeffisientene representerer effekten per standardavviksverdi i stedet for: per originale enhet.

Intercept B_0 = 23.77: Dette er altså modellens prediksjon når alle features er lik 0. Fordi dataene er skalert, betyr dette at bilen har en gjennomsnittlig vekt og akselerasjon, 23.77 er modellens estimerte mpg for en "gjennomsnittlig bil".

B_1 weight = -6.24: Det at det er negativ verdi for beta 1, betyr at økt vekt gir lavere mpg. En økning på ett standardavvik i weight reduserer predikert mpg med 6.24, enklere sagt: tyngre biler bruker mer drivstoff.

B_2 acceleration = 0.99: Det at verdien er positiv betyr at en høyere akselerasjonstid gir høyere mpg. Effekten for accel er mindre nn for weight, siden smp og drivstoffeffektive biler ofte akselerer tregere.

Validation MSE = 18.27: Dette er gjennomsnittlig kvadrert feil på valideringssettet. RMSE (Root mse): sqrt(18.27) = 4.27 Det betyr altså at modellen i snitt bommer med omtrent 4.3 mpg på nye valideringsdata.

For å konkludere: Modellen fanger opp den sterke negative sammenhengen mellom vekt og drivstoffeffektivitet. Med bare to features gir modellen en rimelig feil, men den kan sannsynligvis forbedres ved å inkludere flere relevante trekk.

Optimalisering av koeffisientene: Når `.fit()` blir kalt blir koeffisientene satt til verdiene som minimerer MSE-en på treningsdatasplitten. Hvordan blir dette gjort? For lineær regresjon er det mulig å løse minimering av MSE som et likningsett av flere ukjente, der de ukjente er koeffisientene. Dette kan løses med grunnleggende lineær algebra, der den generelle løsningen kan uttrykkes som et enkelt regnestykke som bruker matrisemultiplikasjon og invertering. Her skiller lineær regresjon seg fra nesten alle andre maskinlæringsmodeller. Dersom modellen vår

blir noe mer komplisert eller inkluderte veldig mange trekk, vil det ikke være mulig å uttrykke koeffisientene som minimerer tapet som et enkelt utrykk, og det brukes som regel iterative numeriske prosesser som *gradientnedstigning* (gradient descent) som det står et bonusavsnitt om i del 2.4 av denne obligen.

1.4 - Trekk-seleksjon (feature selection) [4p, 2b]

Så langt har vi brukt bilenes vekt og akselerasjon til å predikere bensinforbruket, men det originale datasettet inneholder flere trekk som vi kan bruke. Det siste vi skal gjøre med lineær regresjon er å prøve forskjellige trekk. Dette kalles *trekk-seleksjon* (feature selection).

Vi skal se på de fire følgende trekkene:

- `horsepower`: Antall hestekrefter bilene har.
- `model_year`: Året bilene ble produsert.
- `cylinders`: Antall sylindere i motoren til bilene.
- `displacement`: Volumet til motoren.

Oppgave 1.4.1: Tren (på treningsdataene) og evaluer modellen (på valideringsdataene) med trekkene `horsepower` og `model_year` i tillegg til `weight` og `acceleration`. Hvordan forandrer dette resultatet? Prøv å forklare ved å resonnere over hva trekkene representerer og hvordan dette kan relatere til bensinforbruk.

```
# Henter data med de fire feature-ene
# Legger til horsepower og model_year, i tillegg til weight og
acceleration. perform_scaling=True gjør alle features standardizes der
mean=0 og std=1.
mpg_data = get_auto_mpg_data(
    columns_to_include=["weight", "acceleration", "horsepower",
"model_year"],
    perform_scaling=True,
)
x_train = mpg_data["x_train"]
y_train = mpg_data["y_train"]
x_val = mpg_data["x_val"]
y_val = mpg_data["y_val"]

# Trener modellen
model = LinearRegression()

# .fit() funksjonen brukes for å finne koeffisientene som minimerer
mse på treningssettet.
model.fit(x_train, y_train)

# Predikerer på valideringssettet
y_val_predicated = model.predict(x_val)

# Evaluerer med mse
val_mse_4_features = calculate_mse(y_val, y_val_predicated)
```

```python
# Fin print av resultater

print("Model with 4 features, weight, acceleration, horsepower model
year: ")
print("-"*50)
print(f"Validation mse: {val_mse_4_features:.4f}")
print(f"intercept: {model.intercept_:.4f}")
print(f"Coefficients: {model.coef_}")
```

```
Model with 4 features, weight, acceleration, horsepower model year:
--------------------------------------------------
Validation mse: 10.6050
intercept: 23.7702
Coefficients: [-6.0397509   0.59775021  0.75835152  2.98764238]
```

Resonnering 1.4.1:

Sammenlignet med modellen med kun weight og acceleration som hadde en mse rundt 18, så har denne modellen med 4 features blitt bedre. Mse har nå gått ned til 10.61, dette betyr altså at prediksjonene på valideringsdataene er mer presise.

Årsaken til forbedringen er åpenbart at vi nå har to ekstra trekk som er relevante for andel drivstoff bilene bruker.

Horsepower er motorstyrken, der kraftigere motorer bruker mer drivstoff (generelt sett), og dette gir dermed modellen en mer presis informasjon enn kun vekten.

Model_year kan gi oss en bedre teknologisk tankesett. Altså, nyere biler er ofte mer drivstoffeffektive sammenlignet med eldre biler, dette kan handler da om nyfinninger som gjør bilene mer effektive på mange forskjellige punkter. Koeffisienten på 2.99 betyr jo da at nyere biler i gjennomsnitt blir predikert med høyere mpg.

Weight har fortsatt en negativ effekt på -6.04, på grunn av at tyngre biler bruker mer krefter.

Konklusjonen er den at, ved å legge til horsepower og model_year så gir vi modellen mer relevant informasjon som reduserer valideringsfeil. Riktig trekk-seleksjon kan dermed forbedre modellens generaliseringsevne.

Oppgave 1.4.2: Bruk nå `cylinders` og `displacement` i tillegg til de fire andre trekkene til å trene og evaluere modellen. Hvordan blir resultatet nå i forhold til de to forrige tilfellene? Vurder ulike forklaringer på hvorfor resultatet ble slik.

```python
# Nå henter vi data med alle de 6 feature-ene som finnes
mpg_data = get_auto_mpg_data(
    columns_to_include=[
        "cylinders",
        "displacement",
        "horsepower",
        "weight",
        "acceleration",
```

```
        "model_year",
    ],
    perform_scaling=True,
)
x_train = mpg_data["x_train"]
y_train = mpg_data["y_train"]
x_val = mpg_data["x_val"]
y_val = mpg_data["y_val"]

# Trener modellen
model = LinearRegression()
model.fit(x_train, y_train)

# Predikerer på valideringssettet
y_val_predicated = model.predict(x_val)

# Evaluerer mse
val_mse_6_features = calculate_mse(y_val, y_val_predicated)

# Pen utskrift for evalueringen av mse
print("Model with 6 features:")
print("-"*50)
print(f"Validation mse: {val_mse_6_features:.4f}")
print(f"Intercept: {model.intercept_:.4f}")
print(f"Coefficients: {model.coef_}")
```

```
Model with 6 features:
--------------------------------------------------
Validation mse: 11.6115
Intercept: 23.7702
Coefficients: [-0.25651233  1.49670839  0.57287724 -6.9591298
0.77038523  3.04278746]
```

Resonnering 1.4.2:

Når vi legger til cylinders og displacement features-ene i tillegg til de fire andre, så øker validation mse fra 10.6 til 11.6. Dette betyr at modellen faktisk blir dårligere på valideringsdataene.

Cylinders og displacement er faktisk korrelert med weight og horsepower. Flere sylindere og større motorvolum har en sammenheng med høyere vekter og flere hestekrefter. Dette betyr at de nye trekkene ikke nødvendigvis gir oss ny informasjon, men heller overlappende.

Grunnen til at dette er problematisk er at når flere sterkt korrelerte trekk brukes samtidig i en lineær modell, så kan det føre til noe som heter multikollinearitet. Dette gjør at koeffisientene er mer ustabile og kan dermed redusere modellens generaliseriger på nye data som kommer.

Vi kan se at weight har en negativ effekt på -6.96, mens model_year har en positiv effekt på 3.04. Cylinders har en negativ effekt på -0.26, som kan indikere at mye av informasjonen allerede er blitt fanget opp av de andre trekkene.

For å konkludere: Å legge til flere trekk vil ikke nødvendigvis forbedre modellen vår. Selv om cylinders og displacement virker svært relevant og åpenbare å ta med, så gir de faktisk i dette tilfellet lite ny relevant informasjon. Resultatet viser jo at det å legge dem til kan gjøre generaliseringen mindre optimal.

Oppgave 1.4.3 (*Bonus*): Prøv forskjellige kombinasjoner av trekk i datasettet og rapporter den laveste MSE-en du finner på valideringsdataene. Du kan bruke hvilken som helst tilnærming for å prøve ut trekk.

```
from itertools import combinations

# Lager dette til en funksjon for enkelthets skyld
def evaluate_feature_set(feat_name):
    """
    Trener LinearRegression på train-splitt og evaluerer mse på val-
splitt for en gitt liste med feature-navn.
    """
    mpg_data = get_auto_mpg_data(
        columns_to_include=list(feat_name),
        perform_scaling=True,
    )

    x_train = mpg_data["x_train"]
    y_train = mpg_data["y_train"]
    x_val = mpg_data["x_val"]
    y_val = mpg_data["y_val"]

    model = LinearRegression()
    model.fit(x_train, y_train)

    y_val_predicated = model.predict(x_val)
    mse_val = calculate_mse(y_val, y_val_predicated)

    return mse_val

def find_best_feat_comb():
    """
    Bonus oppgave 1.4.3:
    prøv ulike kombinasjoner av features og finn den som gir lavest
mse på val-splitten.

    Vi bruker val-splitt fordi:
    - Train brukes til å lære, fit().
    - Val brukes til å velge modell (feature-set).
    - Hvis vi hadde valgt basert på val, blir val-feilen et
optimistisk (biased) estimat, siden vi optimaliserer mot den.
    """

    # Her er feature trekkene
    all_features = [
```

```python
        "weight",
        "acceleration",
        "horsepower",
        "model_year",
        "cylinders",
        "displacement",
    ]

    best_mse = float("inf")
    best_feature = None

    # For å lagre og printe det senere
    results = []

    # Prøver alle kombinasjoner
    for i in range(1, len(all_features) + 1):
        for feat_comb in combinations(all_features, i):
            mse_val = evaluate_feature_set(feat_comb)

            results.append((mse_val, feat_comb))

            # Sjekker om dette er beste kombinasjonen hittil
            if mse_val < best_mse:
                best_mse = mse_val
                best_feature = feat_comb

    # Sorterer resultatene, fra lavest mse
    results.sort(key=lambda x: x[0])

    # Pen printout
    print("Feature selection (validation mse)")
    print("-"*50)
    print(f"Best comb: {best_feature}")
    print(f"Best validation mse: {best_mse:.4f}")
    print()

    print("Top 10 combinations:")
    for i, (mse, comb) in enumerate(results[:10], start=1):
        print(f"{i:2d}. mse={mse:.4f} features={comb}")

    return best_feature, best_mse

# Kjøring av 1.4.3:
best_features, best_val_mse = find_best_feat_comb()
```

```
Feature selection (validation mse)
--------------------------------------------------
Best comb: ('acceleration', 'model_year', 'cylinders', 'displacement')
Best validation mse: 9.2647

Top 10 combinations:
```

```
 1. mse=9.2647 features=('acceleration', 'model_year', 'cylinders',
'displacement')
 2. mse=9.2891 features=('acceleration', 'model_year', 'displacement')
 3. mse=9.4894 features=('model_year', 'cylinders', 'displacement')
 4. mse=9.5349 features=('model_year', 'displacement')
 5. mse=9.9806 features=('acceleration', 'horsepower', 'model_year',
'cylinders', 'displacement')
 6. mse=10.0730 features=('acceleration', 'horsepower', 'model_year',
'displacement')
 7. mse=10.2089 features=('horsepower', 'model_year', 'cylinders',
'displacement')
 8. mse=10.3007 features=('horsepower', 'model_year', 'displacement')
 9. mse=10.3262 features=('weight', 'model_year')
10. mse=10.3306 features=('weight', 'horsepower', 'model_year')
```

Resonnement for oppgave 1.4.3:

Validation mse: 9.2647 er den laveste validerinsfeilen blant alle de testede kombinasjonene.

Overraskende nok, så er ikke weight og horsepower med i den beste modellen, dette er overraskende på grunn av at det tidligere har vært en sterk predikator. Mye sannsynlig kunne dette bli den beste modellen på grunn av at informasjonen weight gir også blir gitt av de andre feature-ene. Et mulig eksempel er at tyngre biler ofte har større motorvolum og flere sylindere, så disse trekkene kan være sterkt korrelerte, altså et eksempel på multikollinearitet, der flere trekk altså beskriver de samme egenskapene.

Vi ser at feature model_year forekommer i nesten alle kombinasjonene, som tyder på at bilens produksjonsår har mye å si for drivstoffeffektiviteten.

Displacement (motorvolumet til bilen) inngår i mange av kombinasjoenene som sannsynligvis kommer av at større motorvolum ofte innebærer høyere forbruk.

Vi kan igjen se på at mse-verdien vi fikk på 9.2647 nå er enda bedre enn 10.6 vi fikk for 4 features og betydelig bedre enn det første vi testet, nemlig for 2 trekk som ga oss rundt 18 i mse-verdi. Vi kan konkludere med at en mer optimal trekk-seleksjon reduserer prediksjonsfeilen.

Oppgave 1.4.4: (*Bonus*) Bruk den beste modellen (modellen trent på trekkene som gir lavest valideringstap) til å lage prediksjoner for testdataene og rapporter tapet. Dere kan hente testdataene med `mpg_data["x_test"]` og `mpg_data["y_test"]`.

Grunnen til at vi gjør dette er å få et mer realistisk anslag på hvordan den endelige modellen vil prestere på usette datapunkter. Når vi trener mange modeller og velger den med lavest valideringsfeil, blir denne feilen et dårlig estimat for den faktiske generaliseringsfeilen, nettopp fordi modellen er valgt på grunn av sin lave valideringsfeil. Derfor holder vi av et eget testdatasett som ikke brukes i verken prosessering eller trening, slik at dataene er helt nye både for modellen og for dem som utvikler den. På den måten unngår vi modeller som presterer godt på trenings- og valideringsdata, men dårligere ellers - noe som ofte kan skje med kraftigere modeller som dype nevrale nettverk.

```python
def evaluate_best_on_test(best_features):
    """
    Bonus oppgave 1.4.4:
    Denne funksjonen er for å bruke den beste feature-kombinasjonen og
evaluere det på test-splitten.

    Notat:
    - Vi bruker opp val-splitten til modellvalget.
    - test-splittet skal ikke brukes før modellen er ferdig.
    - Test-feilen vil være et bedre estimat på generaliseringsfeil i
dette tilfellet, kontro val-feilene.
    """

    mpg_data = get_auto_mpg_data(
        columns_to_include=list(best_features),
        perform_scaling=True,
    )

    x_train = mpg_data["x_train"]
    y_train = mpg_data["y_train"]
    x_test = mpg_data["x_test"]
    y_test = mpg_data["y_test"]

    model = LinearRegression()
    model.fit(x_train, y_train)

    y_test_predicat = model.predict(x_test)
    mse_test = calculate_mse(y_test, y_test_predicat)

    # printout
    print("\nThe final evaluation on the TEST set")
    print("-"*50)
    print(f"Chosen features: {best_features}")
    print(f"Test mse: {mse_test:.4f}")

    return mse_test

# Kjøring av 1.4.4
mse_test = evaluate_best_on_test(best_features)
```

```
The final evaluation on the TEST set
--------------------------------------------------
Chosen features: ('acceleration', 'model_year', 'cylinders',
'displacement')
Test mse: 14.8718
```

Resonnement for oppgave 1.4.4:

Som vi kan se er trekkene: acceleration, model_year, cylinders og discplacement. Vi fikk en validation mse på 9.2647 i sted og vi får nå en test mse-verdi på 14.8718.

Test-mse verdien er høyere enn validerings mse-verdien vi fikk. Modellen presterer altså dårligere på de helt nye dataene enn på valideringsdataene.

En grunn til at dette skjer kan ha å gjøre med at valideringssettet ble brukt til å velge den beste kombinasjonen. Det er en risiko for at når man tester mange validerings kombinasjoner og at man velger den med hittil lavest verdi at vi har tilpasset oss valideringsdataene for nærme/godt til akkurat dem. Vi kan derfor få et optimistisk estimat.

Siden testsettet er helt nytt og ikke blitt brukt til å lage modellen eller treningen, så kan vi faktisk anta at det gir oss et mye mer realistisk estimat med tanke på hvordan modellen ville prestert på helt nye datapunkter den blir satt til å jobbe med i fremtiden.

Samtidig så kan det faktisk være at modellen ikke var overtilpasset valideringssettet, men at valideringssettet rett og slett var enklere å predikere sammenlignet med testsettet. Den siste muligheten er at det da er en blanding av begge grunnene.

For å konkludere, så forbedret feature selection modellen på valideringsdataene våre, mens testresultatet viser at generaliseringsevnen ikke er like god som valideringsfeilen indikerte. Det er altså veldig viktig å ikke bruke test datasettet til treningen eller modellbyggingen og heller bruke det når alt er ferdig for å finne en mer realistisk evaluering av modellens egenskaper.

Del 2 - Logistisk regresjon [Totalt 13p og 3b]

2.0 - Bakgrunn og datasett [2p, 1b]

I denne delen tar vi for oss binær klassifikasjon med *logistisk regresjon*. Mens lineær regresjon brukes for regresjonsoppgaver der målet er å forutsi reelle tall, brukes logistisk regresjon for klassifikasjonsoppgaver der målet er å tilordne eksempler til *ulike klasser*. Binær klassifikasjon handler om å klassifisere mellom to klasser, for eksempel "ja" eller "nei", eller "0" eller "1".

I binær klassifikasjon er de sanne verdiene y_i enten lik 0 eller lik 1 (dette kan representere binære merkelapper som for eksempel "ikke-hund" og "hund"). Prediksjonene $\hat{y}_i$ er reelle tall mellom 0 og 1.

Selv om logistisk regresjon er en annen modell enn lineær regresjon, er det mange likheter. Det er derfor mulig å gjenbruke noe av koden og metodologien brukt i den første delen.

Datasettet vi skal bruke heter *Spambase*, som er et datasett med trekk (features) fra 4601 e-poster som er markert som spam eller ikke-spam (de sanne verdiene). E-postene har totalt 57 trekk, der vi skal starte med en mindre delmengde av dem. Trekkene beskriver frekvensen av spesifikke ord, tegn og store bokstaver. Vi starter med følgende trekk:

- `word_freq_free`: Frekvensen av ordet `free` i e-posten.
- `char_freq_%24`: Frekvensen av tegnet `$` i e-posten (enkodingen `%24` representerer tegnet `$`).
- `capital_run_length_total`: Summen av antall store bokstaver i e-posten.

I datasettet er "frekvensen" av et ord målt ved antall `100` * `forekomster av ordet` / `totalt antall ord i e-posten` og tilsvarende for frekvensen av tegn.

Datasettet kan lastes med følgende:

```
spam_data = get_spambase_data(
    columns_to_include=["word_freq_free", "char_freq_%24",
"capital_run_length_total"],
    val_ratio=0.2,
    test_ratio=0.2,
    perform_scaling=False,
)
# x_train -> har shape [n, p] der (n e-poster, p features)
x_train = spam_data["x_train"]
# y_train -> har shape [n] der (0 = ikke spam, 1 = spam)
y_train = spam_data["y_train"]
x_val = spam_data["x_val"]
y_val = spam_data["y_val"]

# Henter navnene fra trekkene
feature_names = spam_data["feature_names"]
print("Feature names:", feature_names)

# Henter kolonnen for store bokstaver
capital_letters = x_train[:, 2]

# Finner maksimumsverdien
max_capital_letters = capital_letters.max()

print(f"a) Høyeste antall store bokstaver i en e-post:
{max_capital_letters}")

# Antall spam (1), lager en boolsk array der True for spam, der
np.sum(True) = 1, vektorisert uten løkke.
num_spam = np.sum(y_train == 1)

# Antall ikke-spam (0)
num_not_spam = np.sum(y_train == 0)

print(f"b) Antall spam: {num_spam}")
print(f"   Antall ikke-spam: {num_not_spam}")

# Henter kolonnen for $-frekvensen
dollar_freq = x_train[:, 1]

# Finner maksimum
max_dollar_freq = dollar_freq.max()

print(f"c) Høyeste frekvens av $: {max_dollar_freq}")

# x_train[mask, kolonne] -> Først filtreres radene så hentes
```

```python
kolonnene.

# Lager en maske for ikke-spam
not_spam_mask = (y_train == 0)

# Bruker masken til å hente kun ikke-spam rader
dollar_freq_not_spam = x_train[not_spam_mask, 1]

# Finner maksimum blant disse
max_dollar_not_spam = dollar_freq_not_spam.max()

print(f"d) Høyeste frekvens av $ i ikke-spam: {max_dollar_not_spam}")

# Maske for spam
spam_mask = (y_train == 1)

# Maske for ikke-spam
not_spam_mask = (y_train == 0)

# Henter kapitalbokstav-kolonnen
capital_letters = x_train[:, 2]

# Gjennomsnitt for spam
mean_capital_spam = capital_letters[spam_mask].mean()

# Gjennomsnitt for ikke-spam
mean_capital_not_spam = capital_letters[not_spam_mask].mean()

print("e) Gjennomsnittlig antall store bokstaver i:")
print(f"  Spam: {mean_capital_spam}")
print(f"  Ikke-spam: {mean_capital_not_spam}")
```

```
Feature names: ['word_freq_free', 'char_freq_%24',
'capital_run_length_total']
a) Høyeste antall store bokstaver i en e-post: 15841.0
b) Antall spam: 1094
   Antall ikke-spam: 1666
c) Høyeste frekvens av $: 6.003
d) Høyeste frekvens av $ i ikke-spam: 2.038
e) Gjennomsnittlig antall store bokstaver i:
  Spam: 481.09323583180986
  Ikke-spam: 158.62665066026412
```

Oppgave 2.0: Vi begynner med å undersøke datasettet. I alle deloppgavene under skal vi kun jobbe med dataene i treningssplitten. Navnet på trekkene kan hentes med `spam_data["feature_names"]`, og y-dataene angir spam-e-poster som `1` og ikke-spam som `0`.

- a) Hva er det høyeste antallet store bokstaver i en e-post?
- b) Hvor er antallet spam og ikke-spam e-poster?

- c) Hva er det høyeste frekvensen av tegnet `$` i en e-post?
- d) Hva er det høyeste frekvensen av tegnet `$` i en e-post som *ikke* er spam?
- e) [Bonus] Hva er gjennomsnittlig antall store bokstaver i spam e-poster, og gjennomsnittlig antall store bokstaver i ikke-spam e-poster?

2.1 - Binær kryssentropi [3p]

I regresjon brukte vi tapsfunksjonen MSE, mens vi kommer til å bruke *binær kryssentropi* for logistisk regresjon, som oftest kalt *binary cross entropy* (BCE). Navnet "kryssentropi" kommer fra informasjonsteori, en annen gren innenfor informatikk, men funksjonen passer godt som tapsfunksjon for klassifikasjon.

La oss først se på hvordan og hvorfor BCE fungerer. Vi starter med definisjonen av BCE for et enkelt datapunkt *i*, som er:

$$ \begin{align*} l_{\text{BCE}}(y_i, \hat{y}_i) & = \begin{cases} - \log(1 - \hat{y}_i), & \text{hvis } y_i = 0, \\ - \log(\hat{y}_i), & \text{hvis } y_i = 1. \end{cases} \\[8mm] \text{\textrm{noe som kan også skrives:}} & \\[4mm] &= - \left[y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)\right] \end{align*} $$

Det gjennomsnittlige tapet for et helt datasett og en BCE-tapsfunksjon er dermed:

$$\hat{L}_{\text{BCE}}(\mathrm{y}, \hat{\mathrm{y}}) \quad ¿ \frac{1}{n}\sum_{i=1}^{n} l_{\text{BCE}}(y_i, \hat{y}_i)$$

¿ ¿

La oss kort se på `log()`-delen, altså logaritmen. Generelt sett er logaritmer definert som den inverse funksjonen av potenser, men vi trenger kun å se på tilfellet der inputet er mellom 0 og 1. Da er det to ting å legge merke til: $\log(1)=0$, og $\log(x)$ nærmerer seg minus uendelig når x nærmer seg 0. Dette minuset er opphavet til minuset i starten av formelen for BCE.

Oppgave 2.1.1: Her er noen veiledende spørsmål for å bli kjent med BCE.

- a) For et gitt datapunkt, anta at $y=0$. Hva blir BCE dersom modellen predikerer nøyaktig 0?
- b) Hva omtrent blir BCE dersom modellen predikerer et tall veldig nærme 1?
- c) Anta så at $y=1$. Hva blir BCE dersom modellen predikerer nøyaktig 1?
- d) Hva blir BCE om modellen predikerer et tall veldig nærme 0?
- e) Prøv å beskrive i én setning hvorfor de fire egenskapene over er gunstig for en tapsfunksjon.

Hint: Ukesoppgavene i uke 6 går igjennom BCE i nærmere detalj.

Svar Oppgave 2.1.1:

a)

y = 0 => l = -log(1-y_hatt)

der y=0, predikerer modellen y_hatt = 0

l = -log(1-0) = -log(1) = 0

Svar: 0

b)

der y = 0 og modellen predikerer et tall veldig nærme 1, som 0.9999.

1 - y_hatt er tilnærmet lik 0, og log(0) -> -uendelig Dermed: l = -log(1-y_hatt) -> uendelig

Svar: BCE blir svært stor, den går mot uendelig.

c)

der y = 1 og modellen predikerer y_hatt = 1

l = -log(1) = 0

Svar: 0

d)

Der y=1 og modellen predikerer et tal nær 0, som 0.0001.

log(y_hatt) -> -uendelig når y_hatt -> 0

l = -log(y_hatt) -> uendelig

Svar: BCE blir svært stor, den går mot positiv uendelig.

e)

**Svar: Grunnen til at dette er gunstig har med å gjøre at BCE gir et lavt tap når modellen er sikker og riktig, på den andre siden så straffer den hardt når modellen ikke er sikker og feil.

Oppgave 2.1.2: Implementer funksjonen `calculate_bce()`, som gitt to arrayer av samme lengde regner ut binær kryssentropi (BCE) av de to arrayene. Bruk `np.log()` for logaritmen. Som med MSE kan dere velge om dere vil vektorisere koden eller ikke.

```
def calculate_bce(y_data, predictions):
    """
    Returns the binary cross entropy (BCE) of the input values.

    Args:
        y_data (np.array): [n]-shaped array of true values (zeros or
ones).
        predictions (np.array): [n]-shaped array of predictions
(between zero and one).

    Returns:
        float: The BCE.
    """
    y = np.asarray(y_data, dtype=np.float64)
```

```
    p = np.asarray(predictions, dtype=np.float64)

    # Numerisk stabilitet, vi unngår log(0) ved å bruke .clip()
    eps = 1e-15
    p = np.clip(p, eps, 1.0 - eps)

    # y*log(p) brukes der label er 1 (siden første ledd teller da)
    # (1-y)*log(1-p) brukes når label er 0 (siden andre ledd teller
da)
    # Bruker - foran, for å straffe feil hardt og gi rundt 0 for
korrekt.
    bce_per_point = -(y * np.log(p) + (1.0 - y) * np.log(1.0 - p))

    return float(np.mean(bce_per_point))


test_calculate_bce(input_function=calculate_bce, message_on_pass=True)

# Korrekt prediksjons case
y = np.array([0, 1])
p = np.array([0.0, 1.0])
print(" Test 1 (korrekt):", calculate_bce(y, p))
# Ukorrekt prediksjons case
y = np.array([0, 1])
p = np.array([0.9999, 0.0001])
print(" Test 2 (Ukorrekt):", calculate_bce(y, p))
```

```
Passed: `test_calculate_bce`. All [5/5] tests passed.
 Test 1 (korrekt): 9.992007221626415e-16
 Test 2 (Ukorrekt): 9.210340371976237
```

2.2 - Prediksjon [2p]

Prediksjon er noe forskjellig for lineær regresjon og logistisk regresjon. For lineær regresjon ganget vi vektene med de tilsvarende trekkene og plusset på konstantleddet, som kan gi et tall mellom minus og pluss uendelig. For logistisk regresjon gjør vi først det samme steget, men så putter vi dette tallet inn i en *sigmoid* funksjon som gir et tall mellom 0 og 1.

Sigmoid-funksjonen, som ofte er representert ved σ (det greske symbolet uttalt "sigma"), er definert ved:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Her er e den matematiske konstanten *Eulers konstant*, men i praksis spiller det liten rolle hvilket tall som blir brukt, så det er ikke nødvendig å ha noe særlig forståelse for dette tallet. En ting som kan hjelpe med forståelsen er at e^{-x} går mot 0 når x er veldig høyt og går mot uendelig når x har en høy negativ verdi. Det medfører at $\sigma(x)$ *blir nærme 0 når x har en høy negativ verdi og nærme 1 når* x *har en høy positiv verdi*.

Oppgave 2.2.1: Implementer funksjonen `sigmoid()` som regner ut sigmoid-funksjonen. Bruk `np.exp()` for potensen, der `np.exp(x)` $ = e^x$.

```
def sigmoid(values):
    """
    Calculates the sigmoid function of the input, sigmoid(x) = 1 / (1
+ e^(-x))

    Args:
        values (np.array): [n]-shaped array of values to send to the
sigmoid function.

    Returns:
        np.array: [n]-shaped array of float values corresponding to
the output of the sigmoid function.
    """
    return 1.0 / (1.0 + np.exp(-values))


test_sigmoid(input_function=sigmoid)
print(sigmoid(np.array([-10, 0, 10])))
```

```
[4.53978687e-05 5.00000000e-01 9.99954602e-01]
```

Notat for sigmoid

Prediksjon i logistisk regresjon, forkskjellen fra lineær regresjon. I lineær regresjon så kan y_hatt bli et vilkårlig tall. I logistisk regresjon så beregnes først den lineære kombinasjonen og deretter sender vi dette gjennom sigmoid funksjonen. Sigmoid gjør at output alltid ligger mellom 0 og 1, som da kan tolkes som sannsynligheten for klassen. Vi bruker sigmoid fordi den presser alle tallene inn i 0-1 intervallet, som gjør at output kan tolkes som sannsynlighet. Den er deriverbar, som er viktig for gradient descent og den gir en S-formet kurve. Hvis modellen er sikker på en klasse, så vil sigmoid returnere en verdi rundt 1, rundt 0 ellers. Kan skrives på eksamen som: Sigmoid transformerer en lineær kombinasjon av trekk til en verdi mellom 0 og 1 slik at modellen kan tolkes som et estimert sannsynlighetsverdi for en gitt klasse.

Prediksjon for logistisk regresjon blir altså å først regne ut vektene ganger trekkene pluss konstantleddet, slik som med lineær regresjon, og så putte dette inn i sigmoid-funksjonen. Vektene ganger trekkene pluss konstantleddet blir ofte notert med z, kalt *den vektede summen* (eng: *weighted sum*), og vi kan skrive formelen for prediksjon som:

$$\hat{y} = \sigma(z) = \sigma\left(\sum_{i=1}^{n} \beta_i \cdot x_i + \beta_0\right)$$

Vi kan visualisere logistisk regresjon i følgende diagram:

Oppgave 2.2.2 Implementer funksjonen `logistic_regression_predict()`. Som med lineær regresjon skal metoden ta inn en to-dimensjonal array av dimensjon (n, p), der hver rad er

et datapunkt og kolonnene tilsvarer trekk i inputen, og returnere en vektor av lengde (n) med prediksjoner basert på formelen over. Bruk `sigmoid` implementasjonen deres fra oppgave 2.2.1. **Tips**: Det er mulig å gjenbruke mye av koden fra lineær regresjon, det er kun nødvendig å legge til sigmoid-funksjonen.

```
def logistic_regression_predict(x_data, coefficients):
    """
    Given input data `x_data` of shape [n, p] (`n` datapoints and `p`
features)
    and `coefficients` of shape [p + 1], returns the predictions the
model
    gives as an [n]-length array.

    Args:
        x_data (np.array): [n, p]-shaped array of input data.
        coefficient (np.array): [p + 1]-shaped array of coefficient.
Element number
            zero corresponds to the bias while element one to p
corresponds to
            the weight for feature 1 to p.

    Returns:
        np.array: [n]-shaped data of corresponding predictions.
    """
    # x_data må være et 2D-array: [n, p]
    if x_data.ndim != 2:
        raise ValueError(f"x_data must be a 2D-array of shape [n, p],
got {x_data.shape}!")

    n, p = x_data.shape

    # Koeffisientene må være lengde p+1
    if coefficients.shape[0] != p + 1:
        raise ValueError("coefficients must have length p+1")

    # Splitter bias og weights
    beta0 = coefficients[0]     # bias term
    beta = coefficients[1:]     # Weight vektor

    # kalkulerer vektet sum for alle datapunktene
    z = beta0 + x_data @ beta

    return sigmoid(z)


test_predict_logistic_regression(input_function=logistic_regression_pr
edict)
```

2.3 - Kjøring og evaluering av logistisk regresjon [3p]

Det er på tide å kjøre modellen vår på datasettet og klassifisere e-poster som spam eller ikke-spam.

Oppgave 2.3.1: Tren modellen på treningsdatasettet, prediker på evalueringsdataene og rapporter BCE-en. Dere kan bruke `model = LogisticRegression()` og `model.fit()` som med lineær regresjon. For å få sigmoid-verdiene (prediksjonene før de er rundet av til 0 eller 1) kan dere bruke `model.predict_proba(X=x_val)[:, 1]`, siden `model.predict()` vil returnere klassifikasjonene, altså verdiene etter de er rundet av til 0 eller 1.

```
spam_data = get_spambase_data(
    columns_to_include=["word_freq_free", "char_freq_%24",
"capital_run_length_total"],
    val_ratio=0.2,
    test_ratio=0.2,
    perform_scaling=True,
)
x_train = spam_data["x_train"]
y_train = spam_data["y_train"]
x_val = spam_data["x_val"]
y_val = spam_data["y_val"]

model = LogisticRegression()

# Trene modellen
model.fit(x_train, y_train)

# Predikerer sannsynlighetene. [:, 1] for sannsynlighet for klassen
probabilities_val = model.predict_proba(x_val)[:, 1]

# Calculate BCE
bce_val = calculate_bce(y_val, probabilities_val)

print("Validation BCE:", bce_val)

Validation BCE: 0.4601822243520452
```

Notat til eksamen

Starter med å trene modellen med model.fit(x_train, y_train), som finner vektene for beta som minimerer Binary Cross Entropy.

For den logistiske regresjonen, så brukes predict() for å gi klassifikasjoner og predict_proba() funksjonen gir sannsynlighetene som forekommer.

Bruker BCE til evalueringen siden den straffer feil prediksjoner, er deriverbar og fungerer sammen med sigmoid funksjoenn.

Det neste steget er å evaluere modellen med *nøyaktighet* (accuracy). Nøyaktighet regner ut forholdet mellom korrekt klassifiserte datapunkter og totalt antall datapunkter. Med sanne

verdier $\mathrm{y}=(y_1, y_2, \ldots, y_n)$ og klassifikasjoner $\mathrm{k}=(k_1, k_2, \ldots, k_n)$ (der hver y_i og k_i er enten 0 eller 1), kan vi beregne nøyaktighet som:

$$\text{accuracy}(\mathrm{y}, \mathrm{k})=\frac{1}{n}\sum_{i=1}^{n} I(y_i=k_i)$$

der I er identitetsfunksjonen, som returnerer 1 dersom argumentet er sant, og 0 ellers.

Prediksjonene fra `model.predict_proba()` returnerer verdiene fra sigmoid-funksjonen, men for å regne ut nøyaktighet må verdiene være nøyaktig lik 0 eller 1. Det kan man enkelt få ved å runde av verdier til 0 eller 1 med en terskelverdi, for eksempel 0.5. Dette gjør metoden `model.predict()`.

Oppgave 2.3.2 Implementer funksjonen `calculate_accuracy()` som tar inn to binære arrayer av samme lengde og returnerer nøyaktigheten. Rapporter deretter nøyaktigheten til modellen fra oppgave 2.2.1 på valideringsdataene. Hadde du vært fornøyd med en algoritme for å klassifisere spam med denne nøyaktigheten?

```
def calculate_accuracy(y_data, classifications):
    """
    Calculates accuracy on provided input.

    Args:
        y_data (np.array): [n]-shaped array of true values.
        classifications (_type_): [n]-shaped array of classifications.

    Returns:a
        int: The accuracy for the inputs.
    """
    # Sjekke at array-ene har samme lengde
    assert len(y_data) == len(classifications), "Array must have the
same length"

    # Sammenlignde elementvis, True hvis prediksjonen er korrekt,
False ellers
    correct_prediction = (y_data == classifications)

    # True = 1 og False = 0
    num_correct = np.sum(correct_prediction)

    # Totalt samples
    n = len(y_data)

    # accuracy = correct predictions / total samples
    accuracy = num_correct / n

    return accuracy

test_calculate_accuracy(input_function=calculate_accuracy)
```

```
# Trene modellen
model.fit(x_train, y_train)

# predikerte klasser
y_pred = model.predict(x_val)

# Kalkulere accuracy
accuracy = calculate_accuracy(y_val, y_pred)

print("Validation accuracy:", accuracy)

Validation accuracy: 0.8021739130434783
```

Refleksjonsnotat - Accuracy og evaluering av logistisk regresjon

Accuracy måler hvor stor andel av datapunktene som blir korrekt klassifisert av modellen.

Formelt er accuracy definert som:

$$accuracy(y,k)=\frac{1}{n}\sum_{i=1}^{n} I(Y_i=k_i)$$

hvor:

- y_i er den sanne klassen (0 eller 1)
- k_i er modellens klassifikasjon
- $I(\cdot)$ er en indikatorfunksjon som returnerer 1 dersom uttrykket er sant, ellers 0.

I logisitisk regresjon produserer modellen først sannsynligheter ved hjelp av sigmoid-funksjonen. Disse ligger mellom 0 og 1. For å gjøre klassifikasjon må sannsynlighetene gjøres om til binære verdier. Dette gjøres vanligvis med en terskelverdi på 0.5:

- $\hat{y} \geq 0.5 \Rightarrow 1$
- $\hat{y} < 0.5 \Rightarrow 0$

Metoden model.predic() gjør dette automatisk. Accuracy er en enkel metrikk, men kan være misvisende dersom datasettet er ubalansert. F.eks. hvis 95% av e-postene ikke er spam, så kan en modell alltid predikere "ikke spam" til alltid å få høy accuracy. For spamklassifisering er derfor accuracy nyttig, men bør vurderes i samråd med andre metrikker. Hvis en modell oppnår høy accuracy, som 90%, så kan dette være veldig nyttig, men den kan også bare være veldig god til å klassifisere majoritetsklassen korrekt.

2.4 - Fullt datasett og tolkning [3p, 2b]

Hittil har vi bare brukt 3 av de 57 trekkene i datasettet. Med den generelle implementasjonen av klassen vår kan vi derimot bruke et vilkårlig antall trekk. La oss prøve å trene modellen med alle trekkene.

Oppgave 2.4.1: Tren den logistiske modellen med alle trekkene i datasettet og rapporter nøyaktigheten. Hadde dette vært tilstrekkelig for å bruke modellen til å klassifisere spam i praksis?

```
spam_data = get_spambase_data(
    columns_to_include=None,  # Corresponds to all features
    val_ratio=0.2,
    test_ratio=0.2,
    perform_scaling=True,
)
x_train = spam_data["x_train"]
y_train = spam_data["y_train"]
x_val = spam_data["x_val"]
y_val = spam_data["y_val"]

# Trener logistisk regresjonsmodell på datasettet
model = LogisticRegression()

# fit modellen på trenignsdataen, der modellen lærer koeffisientene
for hver av de 57 trekkene
model.fit(x_train, y_train)

# Generere prediksjonene på valideringsdatasettet
#predict() returnerer klasifikasjonen (0 eller 1)
y_pred = model.predict(x_val)

# Evaluerer model accuracy
accuracy = calculate_accuracy(y_val, y_pred)

print("Validation accuracy with all features:", accuracy)

Validation accuracy with all features: 0.9271739130434783
```

Svar Oppgave 2.4.1

Hadde dette vært tilstrekkelig i praksis?

Svar: Når alle de 57 trekkene i Spambase-datasettet brukes, så får modellen tilgang til mer informasjon om hver e-post. Dette gjør modellen i stand til å identifisere mer komplekse mønstre som skiller spam fra ikke-spam. Dette vil i de fleste tilfeller føre tli høyere presisjon enn for færre trekk.

Selv om nøyaktigheten kan bli høy, så er ikke accuracy alene nok til å si hvor god modellen faktisk oppfører seg i praksis. Grunnen er at accuracy ikke tar hensyn til typer feil som blir gjort av modellen.

I spam filter så vil det være viktigst å ikke ha false positives, siden vi da vil filtrere bort potensielt viktige e-poster og vi kan heller da tolerere litt ekstra spam e-poster som kommer gjennom. På grunn av denne nyansen, så vil det være naturlig for spamklassifiseringssystemer som dette å ha med flere metrikker som: precision, recall og F1-score.

En modell med høy accuracy kan da fortsatt være utilstrekkelig, gitt at det blir gjort klassifikasjonsfeil.

Notat: Accuracy kan være høy selv om modellen faktisk gjør store kritiske feil. For spamfiltre så er precision (e-poster merket spam som er spam) og recall (spam-eposter oppdaget) svært viktige metrikker. Dette er spesielt viktig for False positives (en ekte e-post som klassifiseres som spam), siden det er værre å klassifisere false positives enn å slippe gjennom litt spam.

Oppgave 2.4.2: La oss til slutt reflektere litt over resultatet til modellen.

a) Anta at modellen for spam-klassifisering ble implementert i et ekte e-postfilter. Det er to måter å filtrere e-poster feil på: den ene er å klassifisere spam som ikke-spam, den andre er å klassifisere ikke-spam som spam. Synes du de to typene feil er like dårlige for filteret i praksis, eller vil en type feil være mer problematisk enn den andre? Forklar kort hvorfor du synes det.

Svar 2.4.2 a): I et spamfilter kan modellen gjøre to forskjellige klassifikasjonsfeil, den ene feilen er når en spam e-post blir klassifisert som ikke spam (false-negative) og den andre er når legitim e-post klassifiserer som spam (false positiv).

Den mest problematiske feilen vil være false positive klassifikasjonsfeil, det vil si når en legitim e-post blir klassifisert som spam og filtrert bort. Dette har med at legitime potensielt viktige e-poster aldri kommer frem til mottakeren. Dette kan komme fra arbeidsgiver, leger, banker osv.

False-negatives er derimot mindre alvorlig å slippe gjennom. Mottakeren vil bare måtte manuelt håndtere dem, som er mer friksjon og mindre sømløst, men gir mer kontroll og oversikt.

På grunn av dette vil de fleste spamfiltre minimere sannsynligheten for false positives og heller øke sannsynlighet for false negatives.

b) Hvordan kunne du ha endret modellen slik at man kan prioritere å unngå den ene typen feil over den andre?

Svar 2.4.2 b): Vi kan endre klassifikasjonsterskelen. I logistisk regresjon, så produserer modellen sannsynligheter mellom 0 og 1 ved sigmoid-funksjoenn. E-poster klassifiseres ofte som spam dersom den verdien er større eller lik 0.5 (dette kan justeres som en selv ønsker).

Øker man terskel verdien til typ 0.8, så vil modellen kreve høyere sannsynlighet før den klassifiserer en e-post som spam. Dette reduserer risikoen for false positives, men øker sjansen for false negatives (mer spam slipper gjennom).

Senker man terskelen, til 0,3, vil modellen klassifisere flere e-poster som spam, som reduserer false negatives men øker false positives.

Man kan åpenbart gi større straffer for en av typene under trening, slik at modellen lærer seg å være mer forsiktig for merking av e.poster som spam.

Som tidligere nevnt, så kan man bruke andre metrikker enn accuracy: precision, recall og F1-score.

c) (*Bonus*) Datasettet vårt består av 57 trekk som allerede er manuelt bearbeidet gjennom såkalt *feature engineering*. Har du forslag til andre trekk som kunne vært inkludert for å bedre predikere om e-poster er spam eller ikke?

Svar 2.4.2 c): Så, datasettet som brukes i oppgaven inneholder 57 trekk som er manuelt kontruert gjennom feature engineering, som beskriver frekvens av ord, tegn og bruken av store bokstaver. Det finnes helt klart flere andre typer trekks om kan brukes til å klassifisere spam.

Det mest åpenbare er et trekk som sier noe om avsenders informasjon, ente domene eller IP-addresse og mulig spam koblings historikk.

Spam e-poster har som regel lenker i seg, slik at man blir tatt med til nettsider, så man kan altså analysere og klassifisere basert på antall lenker i mailen.

Ordene som blir brukt i e-posten er særdeles sentrale, man kunne brukt en spam-ordliste, der det er "win", "winner", "free", "lottery", "urgent", "important", "deadline", osv. Det kan også bli flagget på utropstegn og lignende.

Metada om e-posten kan være viktig, som antall mottagere av e-posten for å sjekke om masse-phishing eller reklame, som da typisk er under spam-klasifikasjonen.

Tidligere har det vært svært utbredt å sjekke språklige trekk, altså om det er grammtikkfeil, oversettingsfeil og fraser som ikke passer inn, men med generativ AI, vil dette bare bli mindre og mindre relevant.

For å konkludere, så vil det å ha ekstra trek modellen kan bruke gi mer informasjon om innhold og struktur i e-poster, som kan bidra til bedre prediksjon av om en e-post er spam eller ikke.

Oppgave 2.4.3 (*Bonus*): Tren modellen på forskjellige kombinasjoner av trekk og finn kombinasjonen som gir lavest tap på valideringsdataene. Rapporter nøyaktigheten på testdataene til den kombinasjonen som ga lavest valideringstap.

Oppsummering

I denne obligen har vi sett på lineær og logistisk regresjon, der vi har implementert funksjonalitet som tapsfunksjoner og prediksjon, trent modeller med `sklearn` og tolket resultatet. Selv om både lineær og logistisk regresjon er enkle modeller, er de fortsatt svært effektive og mye brukte. Med god forbehandling av data og trekk kan lineær og logistisk regresjon gi mer enn tilstrekkelig ytelse, og til og med overgå mer komplekse modeller, spesielt med små datasett. De kan også bli brukt for å evaluere andre metoder ved å bruke resultatet som en *baseline* som kan sammenliknes med andre modellers resultat.

I faget IN2160 implementeres lineær og logistisk regresjon helt fra bunnen av, inkludert optimalisering av koeffisientene med gradientnedstigning.